# Greedy algorithms

Suppose it is possible to build a solution through a sequence of partial solutions

- At each step, we focus on one particular partial solution and we attempt to extend that solution
- Ultimately, the partial solutions should lead to a feasible solution which is also optimal

# Definition

A greedy algorithm is an algorithm which has:

- A set of partial solutions from which a solution is built
- An *objective function* which assigns a value to any partial solution

Then given a partial solution, we

- Consider possible extensions of the partial solution
- Discard any extensions which are not feasible
- Choose that extension which minimizes the object function

This continues until some criteria has been reached

# Optimal example

Prim's algorithm is a greedy algorithm:

- Any connected sub-graph of $k$ vertices and $k-1$ edges is a partial solution
- The value to any partial solution is the sum of the weights of the edges

Then given a partial solution, we

- Add that edge which does not create a cycle in the partial solution and which minimizes the increase in the total weight
- We continue building the partial solution until the partial solution has $n$ vertices
- An optimal solution is found

# Optimal example

Dijkstra's algorithm is a greedy algorithm:

- A subset of $k$ vertices and known the minimum distance to all $k$ vertices is a partial solution

Then given a partial solution, we

- Add that edge which is smallest which connects a vertex to which the minimum distance is known and a vertex to which the minimum distance is not known
- We define the distance to that new vertex to be the distance to the known vertex plus the weight of the connecting edge
- We continue building the partial solution until either:
  - The minimum distance to a specific vertex is known, or
  - The minimum distance to all vertices is known
- An optimal solution is found

# Example: Making change

Consider this commonplace example:

- Making the exact change with the minimum number of coins
- Consider the Euro denominations of 1, 2, 5, 10, 20, 50 cents
- Stating with an empty set of coins, add the largest coin possible into the set which does not go over the required amount

# Optimal and sub-optimal examples

Our coin change example is greedy:

- Any subset of $k$ coins is a partial solution
- The value to any partial solution is the sum of the values

Then given a partial solution, we

- Add that coin which maximizes the increase in value without going over the target value

We continue building the set of coins until we have reached the target value

An optimal solution is found with euros and cents, but not with our *quadratic* dumbledore coins

- It fails 29 out of 99 times:

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 12 | 18 | 19 | 22 | 23 | 32 | 41 | 42 | 43 | 48 | 52 | 56 | 61 | 64 | 67 |
| 68 | 70 | 71 | 72 | 73 | 76 | 77 | 80 | 81 | 88 | 90 | 91 | 92 | 97 | |

# Optimal and sub-optimal examples

An implementation of the greedy algorithm is straight-forward:

```
void greedy( int value, int *coins, int *rep, int n ) {
    for ( int i = n - 1; i >= 0; --i ) {
        rep[i] = 0;

        while ( coins[i] <= value ) {
            value -= coins[i];
            ++( rep[i] );              //++rep[i] also works
        }
    }
}
```

# Summary

We have seen the algorithm-design technique, namely greedy algorithms

- For some problems, appropriately-designed greedy algorithms may find either optimal or near-optimal solutions
- For other problems, greedy algorithms may a poor result or even no result at all

Their desirable characteristic is speed